RELATÓRIO DE REVISÃO DO REPOSITÓRIO • PLANO DE CORREÇÕES

EcoSmart Mobile

Atividade 4 — O que corrigir, completar e testar

Persistência local • CRUD • API REST/Firebase • funcionamento online/offline • documentação incremental

Base analisada: github.com/guhfelix/Ecosmart_DMM---Mobile | Branch: main | Consulta: 17 de outubro de 2026

Objetivo: continuar o código já existente em React Native/Expo + TypeScript, sem criar outro aplicativo e sem substituir tecnologias desnecessariamente.

STATUS DA ANÁLISE	INTERPRETAÇÃO
Confirmado no código	O arquivo consultado contém o comportamento descrito. Não significa que tenha sido executado em dispositivo.
Pendente de teste	Precisa de execução local, conexão com Firebase ou verificação em celular/emulador.
Proposta	Ação recomendada; ainda NÃO implementada neste trabalho.

Resumo rápido

O projeto já possui telas de cadastro, histórico, busca, detalhes e exclusão com confirmação, armazenamento via AsyncStorage, serviços Firebase e um servidor REST. As principais pendências da Atividade 4 são: edição de descarte, sincronização offline confiável, confirmação real de sucesso ou erro, segurança de usuários e dados, validação de quantidade e documentação fiel ao funcionamento.

O repositório descreve três aplicativos (Cidadão, Coletor e Admin). O plano abaixo respeita essa implementação real, mas concentra a entrega acadêmica no fluxo do Cidadão e aponta regressões que devem ser verificadas nos demais aplicativos.

Leitura essencial

- P0 — Bloqueio de segurança: impedir escrita aberta no Firestore e uso de credenciais/códigos de demonstração fora de ambiente de testes.
- P1 — Essencial para entregar: implementar Update do descarte; corrigir sincronização falsa; não perder dados offline; tratar erros.
- P2 — Qualidade e documentação: padronizar dados, completar estados de loading, validar em dispositivo e atualizar diagramas e evidências.

Escopo da revisão: inspeção estática de arquivos acessíveis na branch main. Não foram executados npm install, TypeScript, Jest, app Expo nem uma conexão real com Firebase. Os itens listados como risco precisam ser confirmados pelos testes propostos.

1. O que já existe e deve ser preservado

Componente	Situação vista no repositório	Arquivos / notas
Telas do Cidadão	Cadastro de descartes, histórico com busca/filtros, detalhes e exclusão com confirmação.	frontend/ecosmart-cidadao/src/screens/
Orquestração	App.tsx mantém sessão, itens, histórico e operações de criação/exclusão.	frontend/ecosmart-cidadao/App.tsx
Dados locais	AsyncStorage com chaves por perfil e por usuário no Cidadão.	shared/services/storageKeys.ts; App.tsx
Dados remotos	Serviço Cloud Firestore com leituras, gravações e ouvintes em tempo real.	shared/services/firebaseService.ts
API REST	GET/POST/DELETE de descartes e endpoint para baixa de coleta no servidor de sincronização.	scripts/sync-server.js
Conversão JSON	JSON.stringify/parse e normalizadores de descarte entre formatos de apps.	shared/services/syncService.ts; crossAppSync.ts
Conectividade	Hook NetInfo e aviso offline; loading inicial e loading de busca de CEP.	shared/hooks/useNetworkStatus.ts; telas do Cidadão
Cadastro/login	Fluxo de autenticação e cadastro com persistência local da lista de usuários.	shared/services/authService.ts; AuthScreen.tsx

Não é recomendado adicionar Flutter, SQLite ou uma API de demonstração paralela neste estágio. A atividade permite armazenamento local conforme a natureza dos dados e uso de API REST ou Firebase; o ecossistema já apresenta essas opções. O foco é provar consistência e segurança, não multiplicar dependências.

2. Matriz da Atividade 4 — requisito x evidência

Exigência	Avaliação	O que falta para comprovar
Cadastro funcional de usuários	Parcial	Fluxo presente; senha em texto simples e fallback de Google exigem correção/teste.
Cadastro do item principal	Parcial	Formulário e criação presentes; persistência remota não é confirmada.
CRUD completo	Parcial	Create, Read e Delete existem; edição de descarte ainda não está no fluxo do Cidadão.
Persistência local	Parcial	AsyncStorage usado; testar reinício, troca de conta e erro de escrita.
API REST/Firebase	Parcial	Integrações existem; operações silenciam falhas e retornam sucesso indevido.
Objetos ↔ JSON	Presente	Conversão no AsyncStorage, API e normalizadores; padronização merece ajustes.
Cadastro, lista, busca e detalhes	Presente	Telas identificadas; edição e estados de erro precisam de acabamento.
Loading + erros	Parcial	Loading em algumas situações; salvar/sincronizar/excluir não indicam adequadamente falhas.
Online/offline	Parcial	Há detecção e cache, mas confirmação de sync não depende de envio real.
Documentação incremental	Pendente	Exige atualização e evidências correspondentes ao código após correções.

Legenda: “Presente” = evidência estrutural encontrada; não indica teste aprovado. “Parcial” = código existente com pendências. “Pendente” = entrega ainda a preparar.

3. Lista detalhada de correções

Cada item inclui local do código, impacto, menor mudança recomendada e critério objetivo de aceitação. P0 = segurança/bloqueio; P1 = requisito central; P2 = complementação; P3 = opcional.

A4-01 Completar a edição de descartes

Prioridade: P1 | Evidência: Confirmado

Onde foi identificado: DiscardDetailsScreen.tsx (Props só possui onDelete/onBack); RegisterDiscardScreen.tsx (somente novo cadastro); frontend/ecosmart-cidadao/App.tsx (não há rota “edit”).

Problema e impacto: O usuário não consegue alterar categoria, quantidade, observação ou endereço de um registro já salvo; o “Update” do CRUD não fica demonstrado.

Correção mínima proposta: Reutilizar RegisterDiscardScreen em modo edição (prop initialItem / onSave), adicionar botão “Editar” nos detalhes e callback handleUpdateDiscard no App.tsx. Atualizar AsyncStorage e, quando online, enviar a alteração ao mesmo serviço de sincronização. Não recriar tela inteira.

Critério de conclusão / teste: Criar descarte → abrir detalhes → editar → salvar → conferir histórico/detalhes → fechar/reabrir → conferir local e remoto. Exigir que o ID do registro seja mantido.

A4-02 Acrescentar rota REST de atualização por ID

Prioridade: P1 | Evidência: Confirmado

Onde foi identificado: scripts/sync-server.js (rotas GET/POST /api/discards, DELETE /api/discards/:id e /collect, mas sem atualização geral por ID).

Problema e impacto: Sem rota específica de Update, uma edição pode ficar apenas local ou depender de POST de criação com comportamento de substituição implícito.

Correção mínima proposta: Adicionar PUT ou PATCH /api/discards/:id com validação de corpo, ID existente, campos permitidos, usuário autorizado e updatedAt. Não sobrescrever campos de coleta sem regra explícita.

Critério de conclusão / teste: PUT/PATCH retorna 200 com registro atualizado, 400 para corpo inválido, 404 para ID inexistente e 401/403 quando não autorizado; GET posterior mostra alteração.

A4-03 Corrigir “sincronização concluída” sem envio real

Prioridade: P1 | Evidência: Confirmado

Onde foi identificado: shared/services/syncService.ts, processAutoSyncCitizenDiscards() (retira o status offline e incrementa syncedCount sem chamar rede); App.tsx efeito de sincronização.

Problema e impacto: O usuário pode receber notificação de sucesso apesar de o descarte nunca ter chegado ao servidor ou Firestore.

Correção mínima proposta: Tornar o processamento assíncrono; para cada item pendente, tentar gravar no destino configurado, verificar resposta e só então remover o estado pendente. Falha = manter registro pendente e exibir aviso, nunca sucesso.

Critério de conclusão / teste: Ativar modo avião → criar descarte → restaurar Wi-Fi mantendo servidor indisponível: item continua pendente. Ligar servidor → sincronizar → confirmar GET/Firestore e só então exibir sucesso.

A4-04 Persistir fila offline de operações

Prioridade: P1 | Evidência: Confirmado como lacuna no método inspecionado

Onde foi identificado: shared/services/syncService.ts: outboxQueue é array em memória; processAutoSync* apenas altera flags e limpa tipos de mutação.

Problema e impacto: A fila é perdida ao encerrar o app; atualizações e exclusões feitas offline podem não ser transmitidas no próximo acesso.

Correção mínima proposta: Armazenar a fila no AsyncStorage por aplicativo/usuário; cada entrada com id de operação, ação, id do descarte, payload, data e tentativas. Reprocessar ao abrir/reconectar; remover somente após confirmação.

Critério de conclusão / teste: Criar offline → encerrar app → abrir offline → fila persiste → reconectar → servidor recebe uma única vez; simular falha e verificar reenvio sem duplicidade.

Observação: Começar pelo fluxo Cidadão. Depois reaproveitar o padrão para Coletor/Admin apenas onde necessário para não quebrar o monorepo.

A4-05 Garantir exclusão offline e evitar reaparecimento

Prioridade: P1 | Evidência: Risco diretamente sustentado pelo fluxo

Onde foi identificado: App.tsx handleDeleteDiscard remove local antes da confirmação; crossAppSync.deleteDiscard devolve true e ignora falhas HTTP/Firebase; leitura remota pode repor o item.

Problema e impacto: Um descarte excluído pode reaparecer depois de nova consulta, ou o app afirmar exclusão remota inexistente.

Correção mínima proposta: Registrar exclusão pendente (“tombstone”/operação DELETE) no armazenamento local; ocultar imediatamente com indicador pendente; confirmar remoto antes de descartar a pendência. No servidor, DELETE deve respeitar usuário autenticado.

Critério de conclusão / teste: Excluir offline e reiniciar; item continua oculto. Reconectar; confirmar ausência no GET e Firestore. Falhar DELETE de propósito; estado não é reportado como concluído.

A4-06 Corrigir conciliação entre lista local e remota

Prioridade: P1 | Evidência: Confirmado como comportamento de risco

Onde foi identificado: App.tsx (em login/restauração escolhe lista remota não vazia em vez da local; polling adiciona itens e muda status, mas não resolve atualização/exclusão); crossAppSync.fetchAllDiscards agrega caches.

Problema e impacto: Alterações recentes e pendências podem ser sobrescritas na inicialização; exclusões e alterações remotas podem não refletir corretamente.

Correção mínima proposta: Definir regra única: IDs estáveis, updatedAt, estado de sincronização e precedência da fila pendente; mesclar sem descartar mudanças locais. Evitar atualizar cada 1,5 s quando não necessário.

Critério de conclusão / teste: Editar item no dispositivo A → dispositivo B atualiza; desconectar A → editar B → reconectar A e confirmar resolução consistente, sem duplicar/desaparecer.

A4-07 Verificar respostas HTTP e falhas do Firebase

Prioridade: P1 | Evidência: Confirmado

Onde foi identificado: shared/services/crossAppSync.ts: POST/DELETE disparados sem await e com catch vazio; métodos retornam success:true/true. firebaseService.saveDiscardDocument() retorna sucesso mesmo com erro na persistência.

Problema e impacto: Retorno de sucesso não demonstra armazenamento remoto; o critério “tratamento de erros de comunicação” não é cumprido com confiabilidade.

Correção mínima proposta: Usar await na operação escolhida; verificar response.ok, corpo da resposta e timeout. Retornar resultado tipado (localSalvo, remotoSincronizado, erro). Propagar falhas sem descartar o item local.

Critério de conclusão / teste: Desligar servidor / negar acesso ao Firestore: mostrar “salvo localmente, sincronização pendente”; ao restabelecer, confirmar gravação remota e feedback correto.

A4-08 Aprimorar feedback e loading das operações

Prioridade: P2 | Evidência: Confirmado parcialmente

Onde foi identificado: RegisterDiscardScreen.tsx handleSave chama onSave e navega imediatamente; App.tsx addDiscard/exclusão operam async; existe LoadingScreen para inicialização e ActivityIndicator para CEP.

Problema e impacto: Salvar/excluir/sincronizar não possuem estado claro de “em andamento”, e mensagens podem aparecer antes da persistência confirmada.

Correção mínima proposta: Definir saving/deleting/syncing, desabilitar toque duplo, aguardar o resultado da persistência local antes de navegar e exibir mensagens distintas: salvo local, sincronizado, erro.

Critério de conclusão / teste: Simular rede lenta e escrita local com erro; não permitir registro duplicado por clique duplo; loading aparece e desaparece no final.

A4-09 Validar quantidade e definir unidade de medida

Prioridade: P1 | Evidência: Confirmado

Onde foi identificado: RegisterDiscardScreen.tsx verifica apenas texto não vazio; DiscardItem.quantity é string livre em shared/models/index.ts.

Problema e impacto: Entradas como letras ou números negativos podem passar; a atividade original do EcoSmart prevê quantidade e unidade separadas.

Correção mínima proposta: Preservar dados legados e UI, mas introduzir quantidade numérica > 0 e unidade simples (kg, unidade, saco, caixa etc.), com validação. Migrar ou adaptar registros antigos sem apagá-los; no servidor validar coerência.

Critério de conclusão / teste: Tentar vazio, zero, número negativo, letras e valores válidos; campos inválidos não salvam; histórico/detalhes mostram quantidade + unidade de forma clara.

A4-10 Revisar identidade e acesso a dados no Firestore

Prioridade: P0 | Evidência: Confirmado

Onde foi identificado: database/schemas/firestore.rules: coleção /descartes permite read/create/update/delete com true; /usuarios permite leitura pública e criação/atualização sem exigir autenticação para perfil não admin.

Problema e impacto: Qualquer cliente com acesso ao projeto Firebase pode ler/modificar dados além da conta apropriada; invalida isolamento declarado no README.

Correção mínima proposta: Exigir request.auth != null nas mutações; associar IDs aos tokens Firebase; cidadão altera apenas o próprio descarte, coletor apenas campos de coleta autorizados, admin conforme papéis confiáveis. Revisar regras de usuários e notificações.

Critério de conclusão / teste: No emulador de regras: anônimo não altera; cidadão A não modifica usuário B; coletor atualiza apenas coleta; admin autorizado mantém acesso. Só publicar após teste.

Observação: Não confiar apenas em filtro de tela, email ou perfil informado no JSON como controle de autorização.

A4-11 Corrigir cadastro/login e proteção de senha

Prioridade: P0 | Evidência: Confirmado

Onde foi identificado: shared/services/authService.ts: RegisteredUser guarda password; authenticateUser compara texto diretamente; App.tsx persiste registeredUsers no AsyncStorage; existem usuários e senha de demonstração no código.

Problema e impacto: Senhas em texto simples podem ser recuperadas do armazenamento e credenciais fixas não são autenticação segura.

Correção mínima proposta: Para uso real, migrar autenticação por senha para Firebase Authentication; não manter senha em AsyncStorage nem em /usuarios. Manter contas de teste exclusivamente no ambiente de desenvolvimento e documentar a migração.

Critério de conclusão / teste: Criar/login com usuário novo; reiniciar sessão; confirmar que armazenamento de usuários não contém password em texto; testar senha errada e conta de outro usuário.

A4-12 Eliminar autenticação simulada e segredo administrativo no app

Prioridade: P0 | Evidência: Confirmado

Onde foi identificado: shared/services/authService.ts: ADMIN_ACCESS_CODE é constante no código e signInWithGoogle usa identidades locais de fallback quando o login Google falha.

Problema e impacto: Uma falha de login externo pode resultar em sessão “Google” de exemplo; código administrativo distribuído no cliente pode ser descoberto.

Correção mínima proposta: Em ambiente de entrega, falha Google deve retornar erro real; controlar perfil admin no servidor/claims e não por chave embutida no aplicativo. Não ampliar sistema de papéis além do já existente.

Critério de conclusão / teste: Falhar autenticação Google e confirmar que não há login fictício; usuário sem permissão administrativa não entra no Admin.

A4-13 Proteger API REST e validar entrada no backend

Prioridade: P0 | Evidência: Confirmado

Onde foi identificado: scripts/sync-server.js: POST/GET/DELETE /api/discards e POST/PUT /api/users não exigem autenticação na rota; POST aceita campos com defaults.

Problema e impacto: Qualquer cliente que alcance a API pode criar, consultar ou apagar dados de terceiros. Retorno de sucesso não implica autorização.

Correção mínima proposta: Validar token Firebase no servidor para mutações, conferir proprietário/papel e aceitar somente campos permitidos. Para demonstração exclusivamente local, restringir acesso à rede e declarar limite; não chamar de seguro.

Critério de conclusão / teste: POST sem token → 401; DELETE de outro usuário → 403; JSON inválido → 400; ações autorizadas → sucesso. Manter testes do Cidadão, Coletor e Admin.

A4-14 Corrigir confirmação de gravação do servidor JSON

Prioridade: P2 | Evidência: Confirmado

Onde foi identificado: scripts/sync-server.js: saveDb() captura erro de writeFileSync e somente escreve no console; rota POST/DELETE pode responder sucesso mesmo se falhou a gravação do arquivo.

Problema e impacto: Persistência aparente: após reiniciar o servidor, a operação pode desaparecer.

Correção mínima proposta: Fazer saveDb() retornar falha ou lançar exceção e responder 500; em ambiente de testes, simular indisponibilidade da pasta. Não trocar banco do servidor apenas por isso.

Critério de conclusão / teste: Negar permissão de escrita no arquivo de dados e fazer POST: resposta de erro, sem alegar sucesso; restaurar escrita e validar recuperação após reinício.

A4-15 Configurar URL do servidor para dispositivo físico

Prioridade: P2 | Evidência: Confirmado como risco de ambiente

Onde foi identificado: shared/services/crossAppSync.ts usa http://localhost:3333 como fallback, com variáveis EXPO_PUBLIC_SYNC_SERVER_URL / REACT_NATIVE_API_URL.

Problema e impacto: No celular, localhost aponta para o próprio celular, não para o computador do servidor; requisições podem falhar fora do emulador.

Correção mínima proposta: Configurar a variável de ambiente com IP local acessível ou URL HTTPS do backend conforme o cenário; documentar rede Wi-Fi e firewall, sem inserir IP pessoal fixo no repositório.

Critério de conclusão / teste: Executar no Expo Go/dispositivo e conseguir GET /api/health; depois criar descarte e verificar no backend.

A4-16 Normalizar modelos e datas na ida/volta do JSON

Prioridade: P2 | Evidência: Confirmado

Onde foi identificado: shared/models/index.ts define várias visões do descarte; syncService.ts faz conversão com valores padrão; scripts/sync-server.js persiste createdAt/date em formato misto.

Problema e impacto: Campos podem divergir entre apps (type/wasteType, status maiúsculo/minúsculo, date/createdAt), prejudicando edição, ordenação e conciliação.

Correção mínima proposta: Definir um formato canônico para API e datas ISO 8601; normalizar nos limites da aplicação; usar valor padrão apenas em dados legados, não para mascarar campos ausentes na criação.

Critério de conclusão / teste: Gravar e ler descartes com caracteres especiais, diferentes unidades, datas e status; confirmar equivalência dos campos nos três aplicativos.

A4-17 Evitar localização/GPS apresentada como real sem captura

Prioridade: P3 | Evidência: Confirmado

Onde foi identificado: RegisterDiscardScreen.tsx handleUseGps usa DEFAULT_USER_COORDINATES e mensagem de GPS capturado, sem pedido de permissão ou aquisição de posição do aparelho.

Problema e impacto: O usuário pode entender coordenadas fixas de Cáceres como posição do dispositivo. O requisito da Atividade 4 não obriga GPS.

Correção mínima proposta: Como correção mínima, renomear para “Usar localização aproximada de Cáceres” ou remover o rótulo GPS real. Captura efetiva por Expo Location só se a disciplina cobrar esse recurso.

Critério de conclusão / teste: Sem permissão de localização: app não afirma que obteve GPS verdadeiro. Endereço manual funciona offline.

A4-18 Concluir documentação e evidências da Atividade 4

Prioridade: P1 | Evidência: Pendente

Onde foi identificado: docs/ e README.md no repositório; documentação da atividade enviada pelo professor nesta conversa.

Problema e impacto: Se o documento disser “sincronização automática concluída” antes de testes, ele ficará inconsistente com o código real.

Correção mínima proposta: Atualizar os seis itens exigidos (modelo, tecnologias, diagrama, fluxo CRUD, online/offline e organização), registrando explicitamente o que foi testado e o que continua pendente.

Critério de conclusão / teste: Documento contém modelo/diagrama/fluxos com nomes reais dos arquivos e capturas ou registros de testes executados, sem resultados inventados.

A4-19 Reexecutar testes e inspecionar efeitos nos três apps

Prioridade: P1 | Evidência: Pendente de teste

Onde foi identificado: Monorepo com shared/ consumido por Cidadão, Coletor e Admin; README descreve scripts test:all, typecheck:all, test:communication.

Problema e impacto: Uma mudança na serialização, sincronização ou permissões pode quebrar visualização e baixa de coleta no outro aplicativo.

Correção mínima proposta: Rodar checagem TypeScript, Jest e teste de comunicação; complementar com testes manuais de ponta a ponta e regras Firebase. Registrar resultados reais.

Critério de conclusão / teste: Cadastro Cidadão → aparece no Coletor → coletar → status muda no Cidadão → Admin consulta; repetir com falha de rede.

4. Ordem recomendada de execução

Etapa	Tarefas	Por que nesta ordem	Evidência para concluir
0. Segurança / ambiente	A4-10, A4-11, A4-12, A4-13	Restringir escrita e acesso antes de demonstrar dados reais.	Regras testadas, tokens exigidos e sem logins fictícios.
1. CRUD	A4-01, A4-02, A4-09	Fechar o requisito obrigatório de edição e validar o dado.	CRUD íntegro no histórico e no armazenamento local/remoto.
2. Offline e consistência	A4-03, A4-04, A4-05, A4-06, A4-07	Evitar falso sucesso, perda de edição ou ressurreição de exclusão.	Fila persistente, reenvio, retry, respostas verificadas.
3. Experiência e estabilidade	A4-08, A4-14, A4-15, A4-16	Melhorar compreensão e funcionamento nos dispositivos.	Loading/erros, API acessível e JSON consistente.
4. Verificação e entrega	A4-17 (opcional), A4-18, A4-19	Fechar documento com fatos comprovados.	Testes executados, registros e seis itens documentais.

Se o tempo for curto, o mínimo acadêmico deve demonstrar C/R/U/D do Cidadão, armazenamento após reiniciar, serviço remoto funcionando com retorno de sucesso/erro e um cenário offline real. Corrigir exposição de dados antes de publicar ou compartilhar a aplicação fora de testes.

5. Fluxos esperados após as correções

5.1 Cadastro de descarte (Create)

1. Formulário valida categoria, quantidade > 0 e unidade; cria ID estável e identifica usuário autenticado.
2. Escreve localmente (AsyncStorage); somente após confirmação de escrita informa “salvo no dispositivo”.
3. Se conectado, tenta API/Firebase; confirma resposta remota antes de mudar status para sincronizado.
4. Se falhar, mantém operação em fila para novo envio, sem duplicar registros.

5.2 Consulta e busca (Read)

5. Carrega imediatamente o histórico local do usuário autenticado, inclusive quando offline.
6. Quando disponível, consulta remoto com filtro pelo usuário validado no servidor/regras.
7. Concilia por ID/updatedAt, preservando pendências offline; busca e filtros atuam na lista exibida.

5.3 Edição (Update)

8. Usuário acessa detalhes e escolhe Editar; formulário abre preenchido.
9. Após validação, modifica registro com o mesmo ID e atualiza armazenamento local.
10. Cria uma operação UPDATE pendente e transmite quando possível; mantém campos inalterados.

5.4 Exclusão (Delete)

11. Exibe confirmação antes de excluir.
12. Marca exclusão pendente localmente e oculta o item; preserva a operação até confirmação da API/Firebase.
13. Evita reimportação do registro removido de caches antigos; em erro, mantém ação pendente ou oferece tentar novamente.

6. Plano de testes manuais (a executar)

ID	Cenário	Resultado esperado
T01	Cadastrar usuário e reiniciar app	Sessão/conta conforme regra de autenticação; sem senha em texto simples.
T02	Criar descarte online	Histórico/detalhe corretos; confirmação remota verificável.
T03	Criar com internet desligada	Item local persistente e marcado pendente.
T04	Reabrir offline	Item e fila permanecem no celular.
T05	Reconectar com servidor indisponível	Continua pendente, sem falso sucesso.

T06	Restabelecer servidor	Operação enviada uma vez e pendência removida.
T07	Editar dados online e offline	Mesmo ID; campos novos persistem local/remoto.
T08	Excluir offline e reconectar	Não reaparece; DELETE remoto confirmado.
T09	Erro HTTP 400/401/403/500	Mensagem compreensível; sem perda de dados locais.
T10	Duas contas distintas	Conta A não lê/altera registros privados de B.
T11	Coletor confirma coleta	Status se atualiza no Cidadão e segue acessível ao Admin.
T12	Fechar/reabrir servidor REST	Registro confirmado ainda existe no banco do servidor.
T13	Quantidade 0, negativa ou texto	Validação impede cadastro inválido.
T14	Dispositivo físico / rede Wi-Fi	API acessível via endereço configurado, sem depender de localhost no celular.

7. Conteúdo necessário na entrega documental

1) Atualização do Modelo de Dados

Documentar Usuario (id, nome, email, perfil e campos opcionais), DiscardItem/Descarte (id, userId, categoria/type, quantidade, unidade a introduzir, data, observação, status, endereço e syncPending) e tipos equivalentes de Coletor/Admin. Relacionamento: um usuário possui vários descartes. Assinalar campos já existentes vs novas alterações efetivamente concluídas.

2) Tecnologias de Persistência

Informar React Native/Expo + TypeScript, AsyncStorage como cache/sessão/listas locais, Firebase Cloud Firestore como persistência remota e o servidor Node REST com JSON persistido em database/data/ecosmart-live-db.json. Explicar que o arquivo JSON é do servidor, não do celular. SQLite não é obrigatório se a solução local já atende e for demonstrada.

3) Arquitetura de Armazenamento

Desenhar App Cidadão (telas → App.tsx → serviços) → AsyncStorage local; serviços → API REST (scripts/sync-server.js) → arquivo JSON; serviços → FirebaseService → Firestore. Indicar Auth/Firebase Auth e regras de acesso. Diferenciar broadcast local, consulta periódica e persistência real.

4) Fluxo de Dados

Descrever passo a passo Create, Read, Update e Delete, com onde os dados ficam antes/depois de cada operação, quem trata falha e qual status é exibido. Explicar como o JSON representa objetos nas chamadas HTTP e no AsyncStorage.

5) Funcionamento Online/Offline

Detalhar uso de NetInfo, histórico local, pendências, fila, retomada de conexão, reenvio, erro e conflitos. Não afirmar sincronização garantida enquanto não houver POST/PUT/DELETE confirmado e fila persistida.

6) Organização do Código

Registrar caminhos reais: frontend/ecosmart-cidadao/App.tsx e src/screens; shared/models; shared/services/{storageKeys,syncService,crossAppSync,firebaseService,authService}.ts; shared/hooks/useNetworkStatus.ts; scripts/sync-server.js; database/schemas/firestore.rules; docs/. Explicar responsabilidades sem inventar pastas.

8. Esboço textual do diagrama de arquitetura atual

```
App Cidadão (telas React Native / App.tsx)
    ├── AsyncStorage → sessão, usuários locais e descartes por usuário (offline)
    ├── CrossAppSync / SyncService → HTTP JSON → Node.js :3333 → arquivo JSON no servidor
    └── FirebaseService → Cloud Firestore → coleções remotas / regras de segurança
Coletor e Admin → serviços shared → mesmas integrações remotas (com acessos próprios)
```

Nota: este esquema mostra os caminhos identificados. Fluxos automáticos e garantias de sincronização devem ser atualizados no documento somente depois das correções e testes.

9. Não fazer nesta atividade

- Não recriar o aplicativo em Flutter nem adicionar outro banco no celular sem necessidade comprovada.
- Não substituir o backend existente por MockAPI apenas para demonstrar requisições.
- Não reescrever os três aplicativos de uma vez; primeiro corrigir serviços compartilhados com testes de regressão.

- Não afirmar que testes passaram, que o GPS é real ou que a sincronização ocorreu sem evidência observável.

10. Checklist final de aceite

[ ] Cadastro/login validado sem credenciais inseguras fora de teste

[ ] CRUD Cidadão completo com botão Editar e endpoint de atualização

[ ] Quantidade/unidade e demais campos validados

[ ] Persistência local confirmada após fechar e reabrir

[ ] API/Firebase usados com retorno HTTP/erro tratado

[ ] Fila offline persistente e reenvio efetivamente confirmado

[ ] Exclusões offline não reaparecem após reconexão

[ ] Regras Firestore e autorização no backend testadas

[ ] Loading e feedback de operações revisados

[ ] Registros isolados por usuário / acesso permitido

[ ] Comunicação real verificada no dispositivo

[ ] TypeScript, Jest e testes de comunicação executados e registrados

[ ] Documento incremental com os seis tópicos finalizado

11. Fontes e limites da revisão

Fonte principal: https://github.com/guhfelix/Ecosmart_DMM---Mobile (branch main; leitura em 17/10/2026). Arquivos-base conferidos:

Cidadão: frontend/ecosmart-cidadao/App.tsx; src/screens/{RegisterDiscardScreen,DiscardDetailsScreen,HistoryScreen}.tsx.

Compartilhado: shared/models/index.ts; shared/services/{syncService,crossAppSync,firebaseService,authService,storageKeys}.ts; shared/hooks/useNetworkStatus.ts.

Servidor/segurança: scripts/sync-server.js; database/schemas/firestore.rules; README.md.

Limitação: esta revisão não abrangeu todos os arquivos nem comprova o comportamento em tempo de execução; o código pode mudar depois da consulta. Todos os riscos levantados devem ser fechados com testes no ambiente real antes de marcar o requisito como entregue.